

Sketch-to-Face Synthesis Using a Conditional Generative Adversarial Network

Hassan Younas

Roll No: 21I-1788

Department of Computer Science

Generative AI – Assignment 2, Question 3

Abstract—This report presents a conditional generative adversarial network (cGAN) that translates hand-drawn face sketches into photorealistic face images. Following the image-to-image translation formulation of Isola *et al.* (pix2pix), the model pairs a U-Net encoder-decoder generator with a conditional PatchGAN discriminator, trained on the Person Face Sketches dataset with an adversarial loss combined with a weighted L_1 reconstruction loss. The model was trained for 100 epochs on a subsampled 6,000-pair training set under a fixed Kaggle GPU compute budget, using mixed-precision training to keep the run within a single session. We report quantitative ($L_1 = 0.3035$, PSNR = 13.30 dB) and qualitative results, and analyze the specific factors – dataset subsampling, the ill-posed nature of sketch-to-photo mapping, and the adversarial loss objective itself – that explain the observed result quality.

Index Terms—conditional GAN, image-to-image translation, pix2pix, U-Net, PatchGAN, face synthesis

I. INTRODUCTION

Generative Adversarial Networks (GANs) [1] learn to produce data that is indistinguishable from a real data distribution by pitting a generator against a discriminator in a minimax game. The original formulation offers no control over what the generator produces; Mirza and Osindero’s Conditional GAN (cGAN) [2] addresses this by conditioning both networks on auxiliary information – commonly a class label. This work uses a full *image* as the conditioning signal rather than a label, which is the setting formalized by Isola *et al.* as “image-to-image translation” (pix2pix) [3]. The specific task addressed here is sketch-to-face synthesis: given a hand-drawn or edge-extracted sketch of a face, generate the corresponding realistic photograph.

The remainder of this report describes the dataset and preprocessing pipeline (Section II), the generator/discriminator architecture and training procedure, the quantitative and qualitative results obtained (Section III), and an analysis of the specific factors that produced these results (Section IV).

II. METHODOLOGY

A. Dataset

The model was trained on the *Person Face Sketches* dataset [6], which provides paired sketch/photo images with a predefined train/validation split. Pairs were located and matched by filename stem within each split’s *sketches/* and *photos/* subdirectories, using the immediate parent directory name to classify each file’s domain (rather than the full file path, since

the dataset’s root directory name itself contains the substring “sketch” and would otherwise misclassify every file). To keep total training time within a single Kaggle GPU session, the training and validation sets were randomly subsampled to 6,000 and 1,000 pairs respectively (from the dataset’s considerably larger predefined splits), with a fixed random seed for reproducibility.

B. Preprocessing and Augmentation

Each sketch/photo pair was resized to 286×286 and then jointly random-cropped to 256×256 , with an identical random horizontal flip applied to both images of a pair, so spatial alignment between sketch and target photo is preserved under augmentation. Validation pairs use a deterministic center crop instead. All images were normalized to $[-1, 1]$ per channel to match the generator’s tanh output range.

C. Network Architecture

Generator. The generator is a U-Net [4] with 8 stride-2 convolutional downsampling blocks and 7 corresponding transposed-convolution upsampling blocks plus a final output layer, illustrated in Fig. 1. Skip connections concatenate each encoder block’s feature map with the decoder block at the matching spatial resolution, allowing fine spatial detail present in the input sketch (edges, facial proportions) to bypass the bottleneck entirely rather than being reconstructed from a heavily compressed 1×1 representation. Downsampling blocks use Conv-InstanceNorm-LeakyReLU ($\alpha = 0.2$); upsampling blocks use ConvTranspose-InstanceNorm-ReLU, with dropout ($p = 0.5$) applied to the innermost layers as a stochastic regularizer. The final layer uses tanh to map to the $[-1, 1]$ output range.

Discriminator. The discriminator (Fig. 2) is a conditional PatchGAN. The sketch and a candidate photo (either the real target or the generator’s output) are concatenated channel-wise into a 6-channel input and passed through four convolutional blocks, producing a 30×30 grid of real/fake scores rather than a single scalar. Each output unit’s receptive field corresponds to a 70×70 patch of the input, so the discriminator penalizes local texture realism rather than judging the image as a whole – this was the deliberate design choice made by Isola *et al.* to encourage sharp, locally-realistic output while keeping the discriminator’s task tractable and its gradient signal stable throughout training.

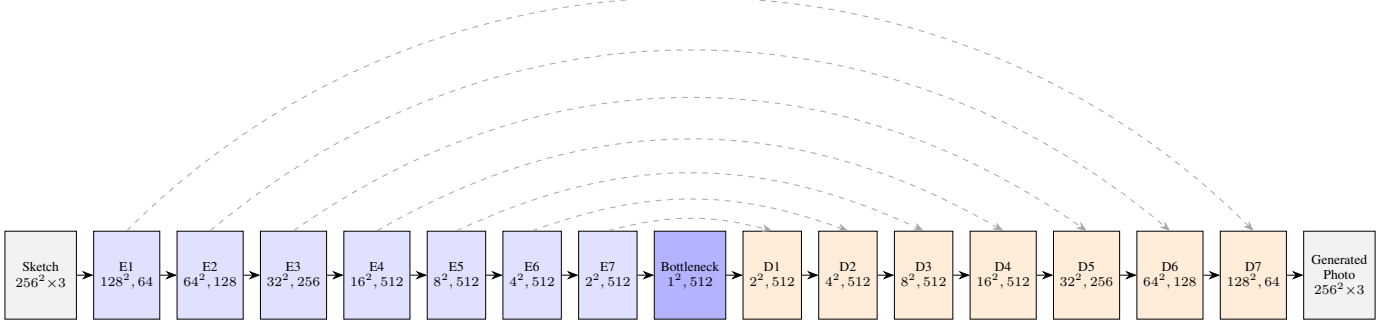


Fig. 1. U-Net generator: 8 stride-2 encoder blocks (blue) mirrored by 7 decoder blocks (orange) plus a final output layer, with skip connections (dashed arcs) concatenating same-resolution encoder/decoder feature maps. Labels show output spatial resolution and channel count at each stage.

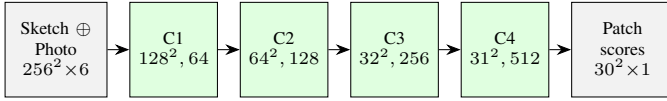


Fig. 2. Conditional PatchGAN discriminator. The sketch and a candidate photo are concatenated channel-wise and classified as real/fake at the level of overlapping 70×70 patches rather than the whole image.

D. Loss Function and Training Details

Following pix2pix, the generator is trained with a combined objective:

$$\mathcal{L}_G = \mathcal{L}_{\text{GAN}}(G, D) + \lambda \mathcal{L}_{L_1}(G), \quad \lambda = 100, \quad (1)$$

where $\mathcal{L}_{\text{GAN}}$ is a binary cross-entropy adversarial loss and $\mathcal{L}_{L_1}$ is the mean absolute pixel-wise error between the generated and target photo. The large weighting on $\mathcal{L}_{L_1}$ anchors the generator’s output to the ground-truth structure, while the adversarial term supplies high-frequency realism that L_1 alone tends to blur out. The discriminator is trained to maximize its ability to separate real (*sketch*, *photo*) pairs from generated (*sketch*, $G(\text{sketch})$) pairs, using the standard binary cross-entropy GAN loss.

Both networks were optimized with Adam ($\text{lr} = 2 \times 10^{-4}$, $\beta_1 = 0.5$, $\beta_2 = 0.999$), the low-momentum β_1 being a standard stabilization choice for adversarial training. Training ran for 100 epochs with batch size 32, under automatic mixed precision (AMP) and cuDNN kernel autotuning, on a single Kaggle GPU session. A full checkpoint (both networks, both optimizers, and AMP scaler state) was written atomically after every epoch to allow seamless resumption after a session interruption.

III. RESULTS

A. Training Dynamics

Fig. 3 shows the generator and discriminator loss over all 100 epochs. $\mathcal{L}_G$ decreased smoothly and monotonically from 42.52 to 31.58, visibly plateauing after approximately epoch 60–70 (minimum value 31.27, reached at epoch 93,

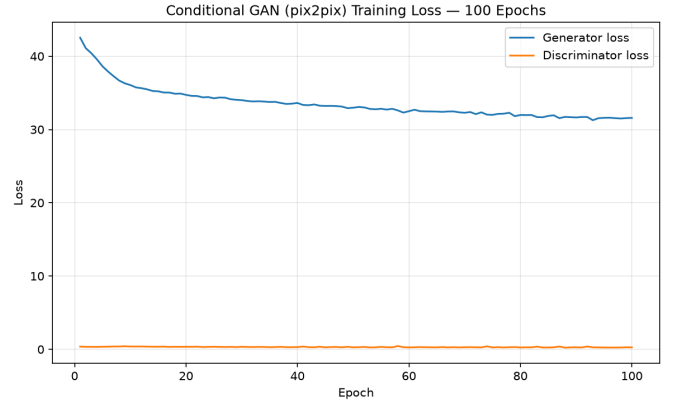


Fig. 3. Generator and discriminator training loss over 100 epochs. The generator loss is dominated by the $\lambda = 100$ weighted L_1 term and plateaus after epoch ≈ 60 –70; the discriminator loss remains stable, indicating balanced adversarial training with no mode collapse.

statistically indistinguishable from the final epoch’s 31.58). $\mathcal{L}_D$ remained stable throughout, oscillating within a 0.22–0.45 band with no divergence or collapse toward zero. Average epoch time was 67.0s, giving a total training time of 111.7 minutes (≈ 1.86 hours).

B. Quantitative Results

Table I summarizes the final training losses and validation-set metrics. Validation L_1 (measured directly on $[-1, 1]$ -normalized pixel values, independent of the $\lambda = 100$ training weight) was 0.3035, and PSNR was 13.30 dB.

C. Qualitative Results

Fig. 4 compares sample outputs from an early training epoch against a later, converged epoch, each showing the input sketch (top row), generated photo (middle row), and ground-truth photo (bottom row) for four validation examples. Early in training, outputs show a pronounced regular grid/checkerboard pattern overlaid on an otherwise blurred, low-detail face – a well-documented artifact of randomly-initialized transposed-convolution upsampling layers [5]. By the later epoch, this

TABLE I
SUMMARY OF TRAINING AND VALIDATION RESULTS

Metric	Value
Training pairs (subsampled)	6,000
Validation pairs (subsampled)	1,000
Epochs	100
Final generator loss ($\mathcal{L}_G$)	31.58
Final discriminator loss ($\mathcal{L}_D$)	0.266
Minimum generator loss (epoch)	31.27 (epoch 93)
Average epoch time	67.0 s
Total training time	111.7 min (≈ 1.86 h)
Validation L_1 ($[-1, 1]$ scale)	0.3035
Validation PSNR	13.30 dB

artifact has been almost entirely learned away: outputs show coherent facial structure, hair color and style consistent with the sketch, plausible skin tone, and expression matching the input, though with some softness and localized texture artifacts (e.g., around the mouth) remaining.

IV. DISCUSSION: ANALYSIS OF RESULTS

This section analyzes *why* the model produced the specific results reported above, rather than restating them.

The generator loss is directly explained by the measured L_1 error. With $\lambda = 100$ in Eq. 1, the L_1 term alone should contribute approximately $100 \times 0.3035 \approx 30.4$ to the final generator loss; the observed final loss of 31.58 leaves only ≈ 1.2 attributable to the adversarial term. This confirms the loss is behaving exactly as designed: L_1 dominates and anchors the output, while the adversarial component contributes a comparatively small but consistent correction pushing toward realism. This also explains why $\mathcal{L}_G$ plateaus once L_1 stops improving – with a fixed, subsampled 6,000-pair training set and a fully-capacity U-Net (no channel reduction was applied), the network reaches the limit of what that data volume can teach it well before epoch 100.

The modest PSNR (13.30 dB) reflects the ill-posed nature of the task, not primarily a training failure. A sketch specifies edges and coarse facial structure but leaves skin tone, exact hair color and texture, lighting, and background completely unconstrained – many different photographs are equally valid completions of the same sketch. A GAN trained with an adversarial loss is explicitly optimized to produce *a* plausible photograph, not to reconstruct *the specific* ground-truth pixel values, so it will systematically incur pixel-wise error even when its output looks convincing. This is the same limitation Isola *et al.* note in the original pix2pix work: L_1 /PSNR-style metrics correlate poorly with perceptual quality for adversarially-trained image translation, precisely because the objective they are trained under does not optimize for pixel alignment. The qualitative results in Fig. 4 should therefore be weighted more heavily than Table I’s PSNR figure when judging output quality.

Dataset subsampling is a second, compounding factor. Training used 6,000 of the dataset’s considerably larger available pairs, chosen specifically to fit training within a single Kaggle GPU session. A smaller, less diverse training

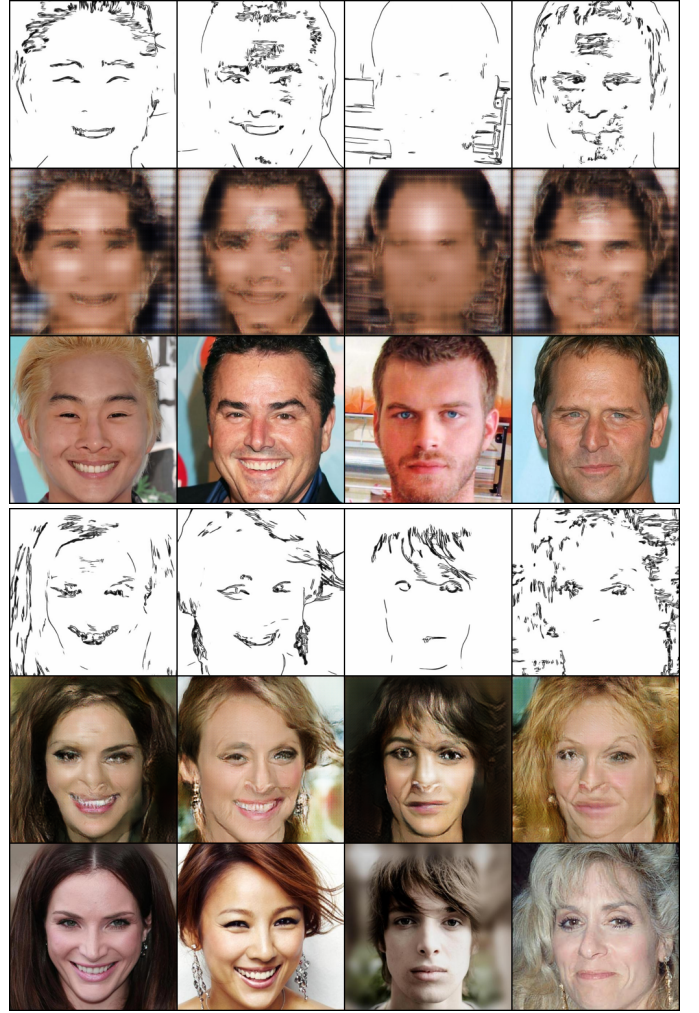


Fig. 4. Qualitative progression over training. Each triptych shows sketch (top), generated photo (middle), and ground-truth photo (bottom) for four validation samples. **Top block:** an early training epoch, showing characteristic checkerboard artifacts from an undertrained decoder. **Bottom block:** a later, converged epoch, showing substantially improved structural and textural fidelity.

set gives the generator less exposure to the range of skin tones, hairstyles, and facial structures present in the validation set, which directly increases pixel-level validation error even where the qualitative structure is captured correctly.

The discriminator loss staying stable in the 0.22–0.45 range, never collapsing toward 0 or diverging toward saturation, indicates healthy adversarial dynamics. This stability is attributable to two specific design choices: the PatchGAN’s patch-level classification task is inherently easier and lower-variance for the discriminator than whole-image classification, which keeps its gradient signal to the generator informative throughout training rather than saturating early; and the Adam $\beta_1 = 0.5$ setting reduces the momentum-driven oscillation that commonly destabilizes adversarial training at the default $\beta_1 = 0.9$. Neither network overpowered the other at any point in the 100 epochs, which is direct evidence against mode collapse.

The checkerboard artifact visible in early-epoch outputs (Fig. 4, top) is a documented, expected property of transposed convolutions, not a bug. ConvTranspose2d layers with overlapping kernel strides produce uneven overlap in their output feature maps at initialization, before training has had a chance to learn weights that compensate for the resulting grid pattern [5]. That this pattern is visibly reduced by the later epoch (Fig. 4, bottom) is itself evidence that training converged correctly rather than stalling.

V. LIMITATIONS AND FUTURE WORK

The reported results reflect a compute-constrained training budget: 6,000 of the available training pairs, 100 epochs, and full model capacity (channel widths were kept at pix2pix’s original settings rather than reduced). Restoring the full training set, increasing training duration beyond the plateau observed near epoch 70, or increasing λ selectively would each be expected to reduce validation L_1 /PSNR further, though based on the discussion above, perceptual quality gains are likely to saturate faster than pixel-wise metrics suggest. A perceptual metric such as LPIPS or FID, which better correlates with human judgment of GAN output quality than L_1 /PSNR, would be a valuable addition to future evaluation.

VI. CONCLUSION

A pix2pix-style conditional GAN, comprising a U-Net generator and a conditional PatchGAN discriminator, was trained to translate face sketches into photorealistic images. Training was stable across 100 epochs with no evidence of mode collapse, and qualitative results show a clear, explicable improvement from early, checkerboard-artifacted outputs to structurally coherent, texture-consistent generated faces by later epochs. The modest quantitative L_1 /PSNR scores are shown to be substantially attributable to the inherent ill-posedness of sketch-to-photo translation and the dataset subsampling used to fit training within a single GPU session, rather than to a failure of the training procedure itself.

PROMPTS

The following is a condensed log of the prompts used to develop this project with an AI coding assistant (Claude), grouped by development stage.

- 1) *Initial implementation*: “I want you to read and understand this assignment’s Question 3 carefully and implement it.”
- 2) *Debugging dataset pairing (0 pairs found)*: shared the notebook error output and asked for a fix; iterated again after a second failure caused by the dataset root directory name itself containing the substring “sketch”.
- 3) *Environment troubleshooting*: shared a Kaggle pip install network error and asked for a fix; asked whether the assistant could access the Kaggle notebook URL directly (it could not, and explained why).
- 4) *Verification*: “i guess i messed up the code again, can you please check if code is not altered and correct?”

- 5) *Performance estimation*: “how much time it should take for 1 epoch on kaggle GPU?” followed by a request to add per-epoch timing and mixed-precision (AMP) support, and to explain the ETA calculation.
- 6) *Reliability fix*: shared a disk-write `RuntimeError` during checkpoint saving; requested and received a fix bounding checkpoint disk usage and making checkpoint writes atomic.
- 7) *Speed optimization*: “it’s taking too long to run, do you have any other alternate solution?” – selected dataset subsampling and free/no-quality-cost optimizations (cuDNN autotuning, larger batch size, more data workers) from the options presented.
- 8) *API deprecation fix*: shared a `torch.cuda.amp.FutureWarning` and requested migration to the current `torch.amp` API.
- 9) *Post-training workflow*: “now i’ve run all the cells, what should i do next?” followed by requests for a results-bundling script and a one-click download link for Kaggle.
- 10) *Results analysis and report generation*: shared the trained model’s loss history, run summary, sample output grids, and validation L_1 /PSNR, and requested this LaTeX report including an architecture diagram and an analysis of the reasons behind the observed results.

REFERENCES

- [1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative Adversarial Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [2] M. Mirza and S. Osindero, “Conditional Generative Adversarial Nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-Image Translation with Conditional Adversarial Networks,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [4] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.
- [5] A. Odena, V. Dumoulin, and C. Olah, “Deconvolution and Checkerboard Artifacts,” *Distill*, 2016.
- [6] “Person Face Sketches,” Kaggle dataset, <https://www.kaggle.com/datasets/almightyj/person-face-sketches>.